ACADEMIC PROJECT PRESENTATION

Pulse Monitor

ESP32-based heart-rate & SpO_2 monitoring with cloud logging and AI-assisted measurement review

Wokwi simulation · ThingSpeak cloud · Python bridge · AI service · SQLite · web dashboard

Presented by: _____

Guide: _____

Department of _____ · Oct 2026



Agenda

1. Problem & objectives
2. System architecture at a glance
3. Components — role, objective & how each works
4. Cloud path: ThingSpeak → bridge → local API
5. What the AI service does — and what it returned
6. Dashboard & end-to-end walkthrough of one reading
7. Verification results
8. Limitations, safety & future work

At a glance

1 simulation source · 1 cloud log

3 local services · 1 database

2-stage AI review layer

every claim verified on the running system

Why this project: problem & objectives

The problem

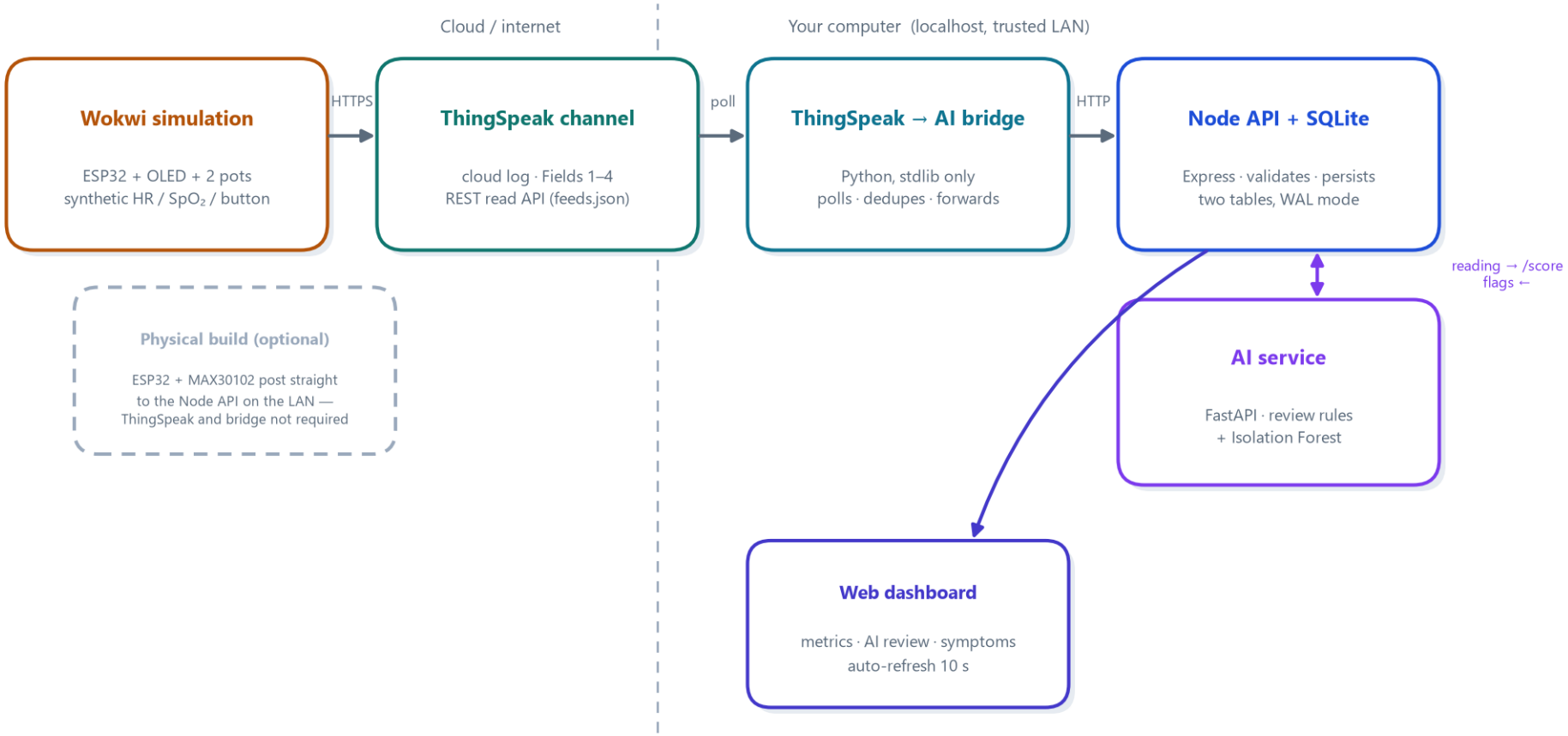
- Continuous vitals logging is easy to start but hard to finish — data ends up stranded in one cloud dashboard.
- No local record, no review layer, and no context for what the numbers mean.
- Early prototypes often imply medical accuracy they do not have.

Objectives

- Acquire pulse + SpO_2 estimates optically (MAX30102 on an ESP32).
- Reject invalid measurement windows before anything is uploaded.
- Persist every valid reading locally (SQLite) and visualise it live.
- Let the user log symptoms so readings have context.
- Publish to ThingSpeak for cloud storage and MATLAB analysis.
- Add an honest AI layer: transparent rules + unsupervised anomaly scoring.

Research prototype — not a medical device. It does not diagnose anything, and it is not a substitute for professional care.

System architecture



Two ingestion paths converge on one local API: the cloud path (simulation → ThingSpeak → bridge) and a direct LAN path (physical board). Every stored reading is scored by the AI service, and the dashboard reads only from the local API — one source of truth.

Components at a glance — each has one clear job

MAX30102 sensor

Optical PPG: red + IR LEDs and a photodiode yield pulse and SpO_2 estimates.

ESP32 DevKit

Reads the sensor over I^2C, validates windows, uploads over Wi-Fi.

Wokwi simulation

ESP32 + OLED + pots reproduce the firmware without hardware.

ThingSpeak channel

Cloud log (Fields 1–4) + REST read API for the PC.

Bridge (Python)

Polls the channel, dedupes entries, forwards to the local API.

Node API + SQLite

Validation, persistence, AI calls, REST for the dashboard.

AI service (FastAPI)

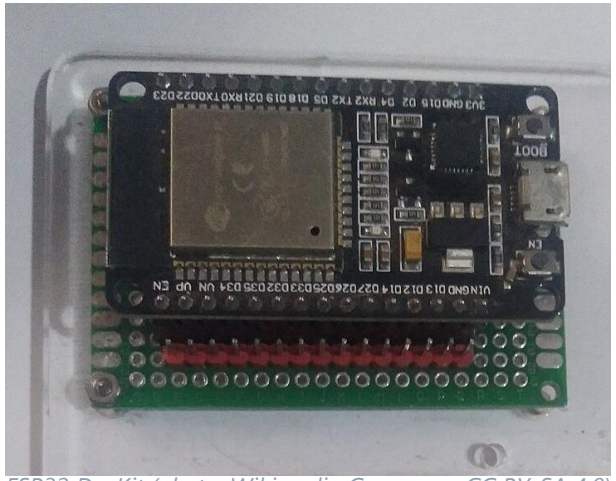
Review rules + Isolation Forest anomaly scoring.

Dashboard (web)

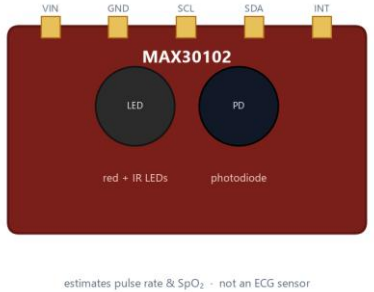
Metrics, latest AI review, flags table, symptom journal.

Core hardware — MAX30102 sensor & ESP32

- MAX30102 — objective: acquire the pulse waveform optically.
  - Red + IR LEDs shine into the skin; a photodiode measures the reflected light.
  - Pulse rate and SpO_2 are estimated from the light ratios (PPG).
  - It is not an ECG — beat shape / electrical activity are not captured.
- ESP32 — objective: turn raw samples into trustworthy readings.
  - Reads the sensor over I^2C in 100-sample windows (SparkFun MAX3010x library).
  - Uploads only when a window is valid: pulse and SpO_2 found, ranges plausible.
  - Adds a signal-quality heuristic and a beat-interval variability flag — explicitly not arrhythmia detection.

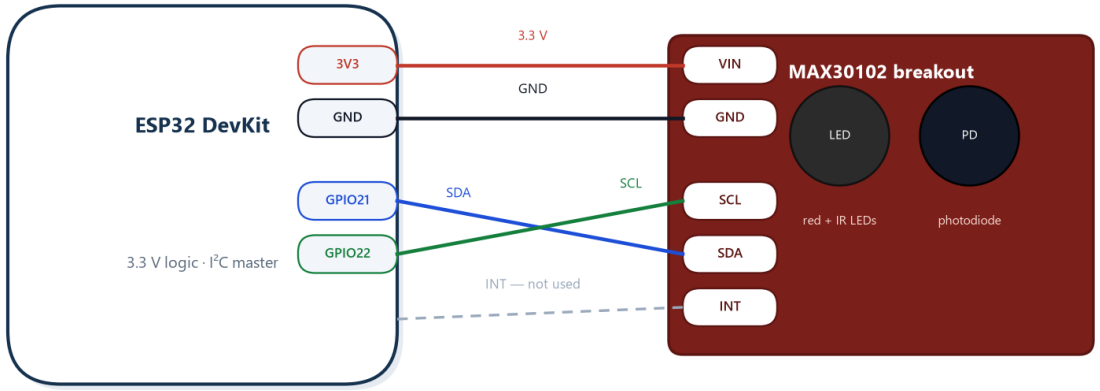


ESP32 DevKit (photo: Wikimedia Commons, CC BY-SA 4.0)



MAX30102 module — red + IR LEDs and photodiode (illustration)

Interface — wiring the sensor to the board



I²C address 0x57 · 3.3 V supply only · short wires, keep still

- I²C at 3.3 V: SDA → GPIO21, SCL → GPIO22; sensor address 0x57.
- The same wiring is mirrored one-to-one inside the Wokwi simulation.
- INT pin exists but is not used in this prototype.
- Practical quality note: short wires, gentle contact with the fingertip, keep still during measurement.

Powering the module from 5 V or swapping SDA/SCL are the two most common wiring mistakes — the sensor simply returns zeros when it is unhappy.

The Wokwi simulation — developing without hardware

- Objective: exercise the complete cloud + AI + dashboard pipeline before real hardware exists.
- Contains: ESP32 + SSD1306 OLED + two potentiometers + one push button.
- Posts to ThingSpeak every ~20 s, respecting the free-tier limit (≥ 15 s).
- The OLED shows live HR, SpO_2, quality and Wi-Fi status during the run.

Control	Effect
HR potentiometer	heart rate 45–150 bpm → Field 1
SpO_2 potentiometer	85–100 % → Field 2
Push button	manual / test flag → Field 4
OLED display	live status on the virtual device

Honest note: the simulation validates the software pipeline — not sensor accuracy. Its values are synthetic by design.

Cloud logging — ThingSpeak channel

Field	Meaning	Values
Field 1	heart rate	bpm
Field 2	SpO_2	%
Field 3	signal quality	0–100 %
Field 4	manual / test flag	0 or 1

- Objective: durable cloud storage, MATLAB visualisations, and a pull point the PC can read.
- Working: every accepted reading becomes one timestamped entry; the read API (feeds.json) returns them as JSON.
- Only the newest entries matter for live use — the bridge reads the last 100 and picks up where it left off.
- The write key lives only in the firmware; the channel is public for this project.

ThingSpeak → local API bridge (Python)

- Polls feeds.json every 10 s and sorts entries by ID.
- Skips everything already processed — the state file remembers the last entry ID, so no duplicates and no silent replays.
- Validates numeric fields; a broken entry is retried, never skipped.
- Forwards each new entry to POST /api/readings and prints the AI result to the console.

Why this exists: Wokwi runs in the cloud and cannot reach 127.0.0.1 on your PC. So instead of the device pushing to the laptop, the laptop pulls from the cloud.

- Poll the channel (≤ 100 entries).
- Keep only IDs newer than the state file.
- Forward to the local API and check saved=True.
- Advance the state file only after a successful save.

```
ThingSpeak entry 3 forwarded  |  HR=45 bpm  SpO2=100%
API saved=True  |  AI flags=['heart_rate_outside_configured_review_band']
```

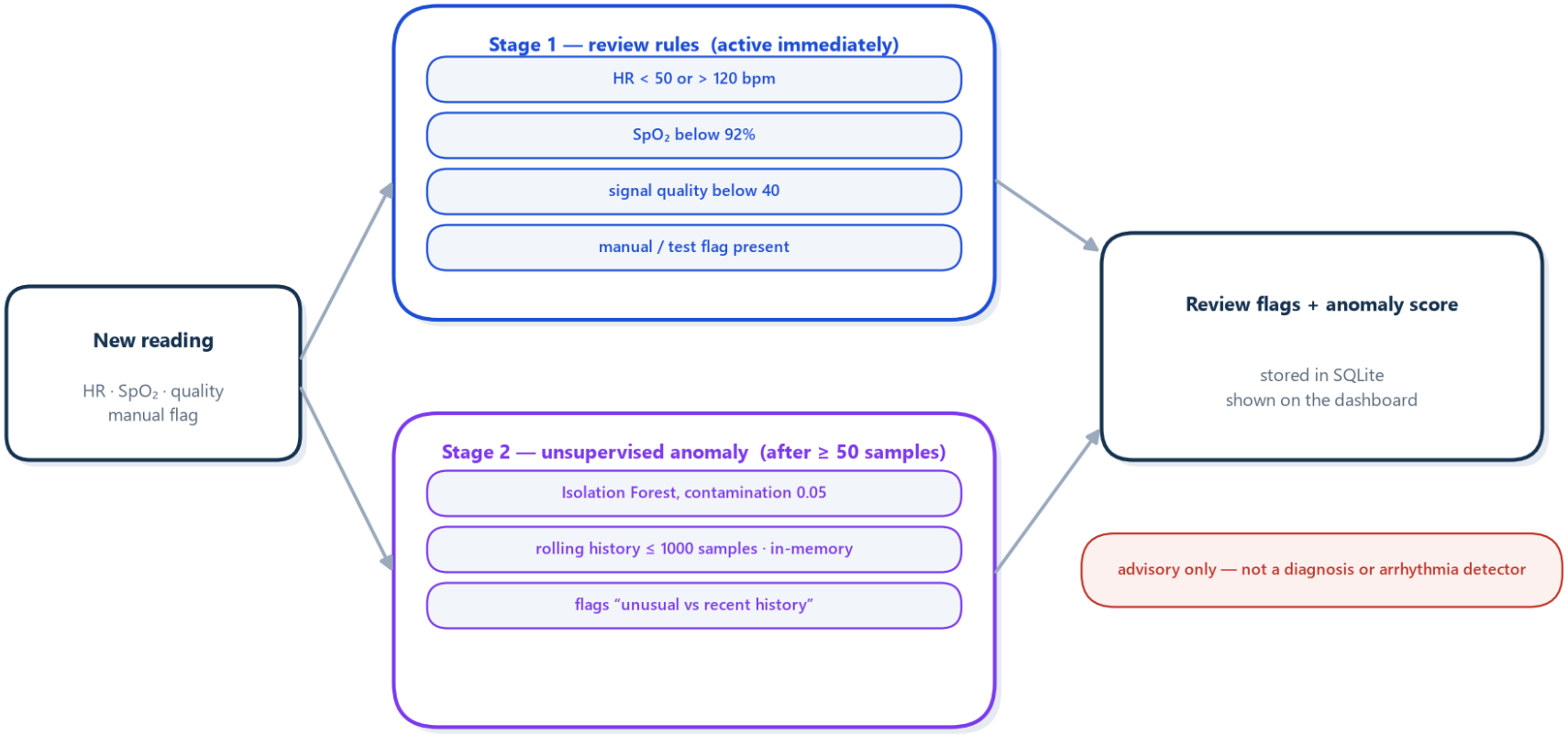
Config: THINGSPEAK_CHANNEL_ID (+ optional READ_API_KEY) · first run processes only the latest entry; full history replay is opt-in via REPLAY_HISTORY_ON_FIRST_RUN.

Local server — Node API + SQLite

Endpoint	Job
POST /api/readings	validate → store → AI score → save flags
GET /api/readings	recent readings with their AI flags
POST/GET /api/symptoms	symptom journal entries
GET /api/health	service status

- Range validation (HR 25–240, SpO_2 70–100, quality 0–100) — bad data never reaches the database.
- If AI_SERVICE_URL is configured, every reading is scored and its flags stored alongside.
- SQLite in WAL mode with two tables (readings, symptoms); the database is created automatically and older files upgrade in place.
- Bound to the local network only — nothing is exposed to the internet.

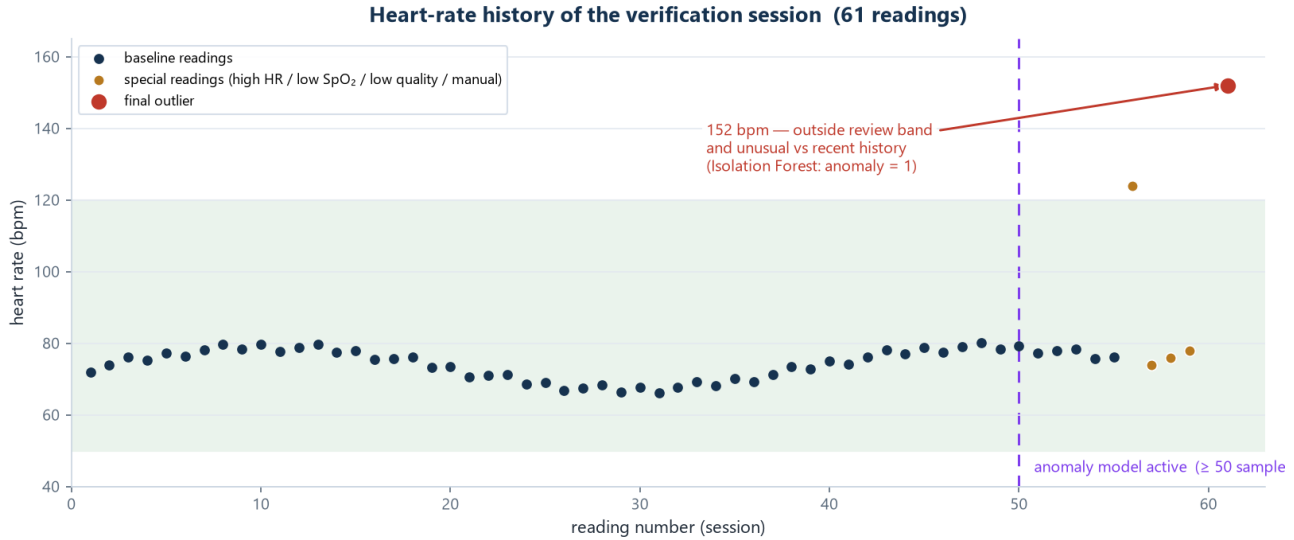
What the AI service does



- A local FastAPI microservice (POST /score) — no health data leaves the computer.
- Stage 1 rules are deterministic and explainable; Stage 2 learns what is normal for this device only, in memory, and needs ≥ 50 samples before it says anything.
- The response itself carries the disclaimer: experimental screening aid, not validated for diagnosis or arrhythmia detection.

What the AI returned — real results

- Live example — ThingSpeak entry 3: HR 45 bpm → flag: “heart rate outside configured review band”; stored and shown on the dashboard.
- Verification session (61 readings): one HR 152 bpm outlier received 5 flags, including “unusual relative to recent device history” (Isolation Forest, anomaly = 1).
- Every flag is a prompt to review context — never a conclusion.

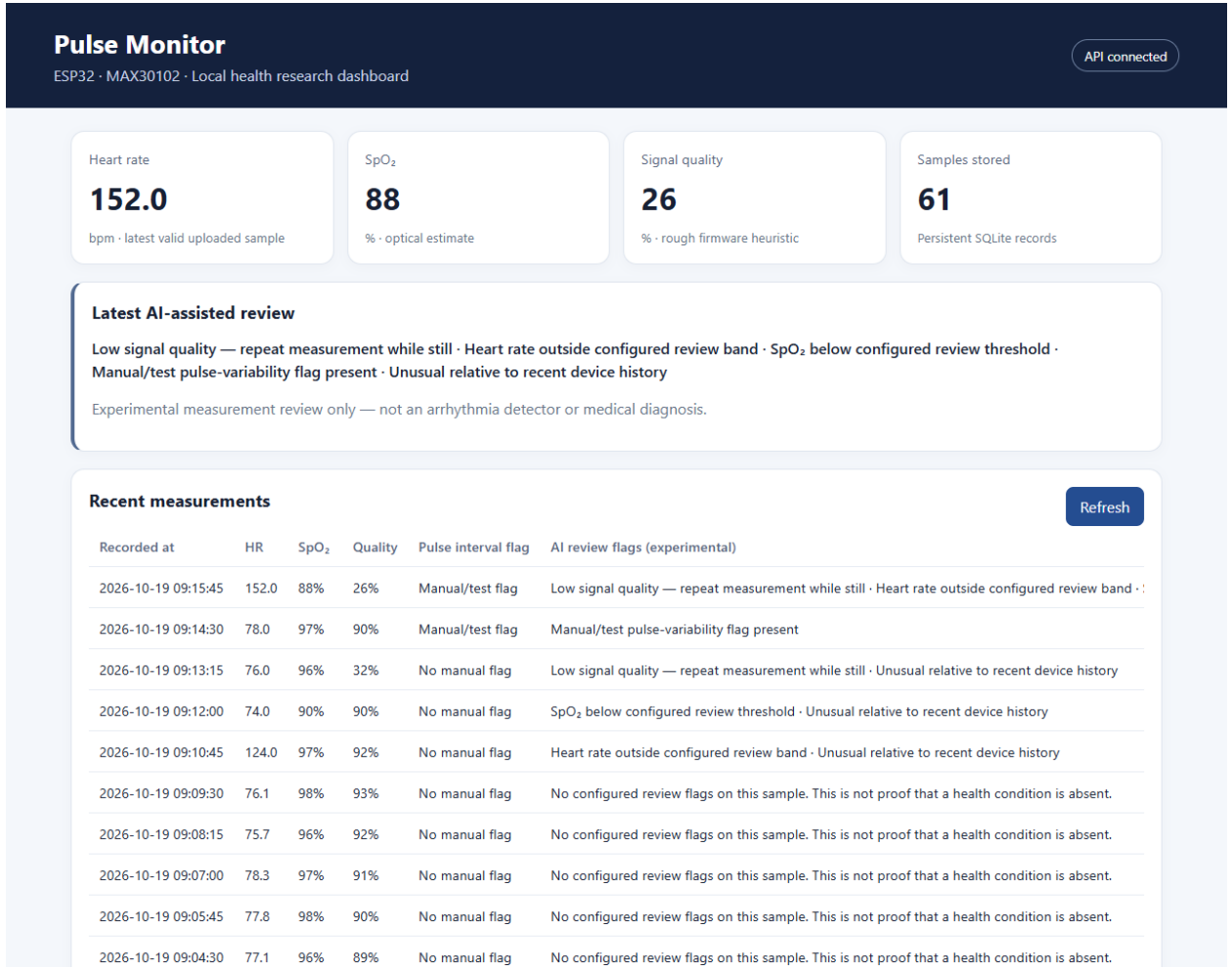


Verification run: baseline, planned special readings and the flagged outlier.

```
“reviewFlags”: [
   “low_signal_quality”,
   “heart_rate_outside_configured_review_band”,
   “spo2_below_configured_review_threshold”,
   “pulse_interval_variability_flag_from_firmware”,
   “unusual_relative_to_recent_device_history”
],  “unsupervisedAnomaly”: 1
```

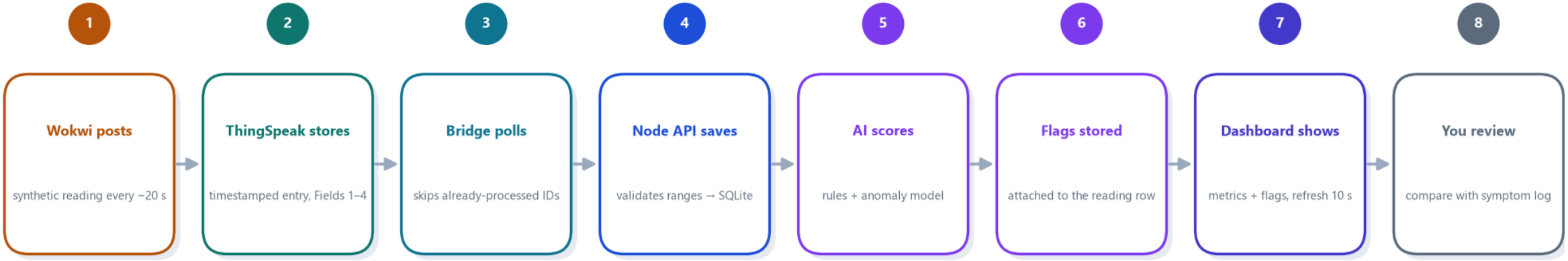
The stored AI response for the outlier — verbatim.

The dashboard — where the user meets the data



- Metric cards: latest heart rate, SpO_2, signal quality, sample count.
- “Latest AI-assisted review” panel: plain-language flags for the newest reading.
- Measurements table: every reading with its AI flags — plus the symptom journal below.
- Auto-refreshes every 10 s; open it from any device on the LAN.
- Disclaimers are visible in the interface itself, not buried in documentation.

One reading, end to end



entry 3 · HR 45.0 bpm · SpO_2 100 % · quality 95 → forwarded by the bridge → saved by the API → AI flag: heart_rate_outside_configured_review_band → dashboard shows the row with the flag in plain language.

- If the API call fails, the bridge retries — the entry is never marked processed until the save succeeds.
- The AI model's history is kept in memory, so a service restart simply starts learning again.

Verification results — observed on the running system

Test	Result
Live chain: ThingSpeak entry → dashboard	one real reading stored, scored and displayed — end to end in under one poll cycle ✓
Bridge duplicate safety	restarts and re-polls produced no duplicate rows ✓
Broken-entry handling	corrupt entries were retried; the state file never advanced past unsaved data ✓
First-run policy	only the latest existing entry processed by default; replay is opt-in ✓
AI anomaly stage	outlier flagged after 61 in-service samples (anomaly = 1) ✓
Dashboard QA	0 console errors, 0 failed network requests during the full session ✓
Integration defects found & fixed	AI service startup, console encoding, SpO_2/quality unit display ✓

Every row above was observed on the actual pipeline against the project's real ThingSpeak channel — not estimated.

Limitations, safety & what comes next

Limitations — stated honestly

- Sensor data in the simulation is synthetic by design.
- An optical PPG sensor is not an ECG and cannot see beat morphology.
- The anomaly model is unsupervised and in-memory (it resets on restart).
- Thresholds are prototype choices, not clinical values; symptoms are self-reported.
- Local use only: no accounts, no encryption at rest.

Safety rules & next steps

- Frame it as a logging and review aid — never as a diagnosis; keep disclaimers visible.
- Keep API keys private; rotate any key that is ever shared in a screenshot.
- Next: collect real labelled data with an ECG reference.
- Then: validated metrics (sensitivity / specificity) and a supervised model trained on that data.
- Later: persistent training, alerts, multi-user access, documented protocol.

Conclusion

- A complete pipeline was built and verified end to end: simulation → cloud → bridge → API → AI review → SQLite → dashboard.
- Each component has one clear job, and every failure mode found during testing was fixed and re-verified.
- The AI layer adds transparent review flags plus an experimental anomaly signal — framed as an aid, not a diagnosis.
- The system is ready for its next phase: real sensor data and formal validation.

Thank you — questions?